

Informe de vulnerabilidades y soluciones

Proyecto: JR_M26_AgenciaViajes / Turistea

Fecha del informe: 2026-05-08

Alcance: revision estatica de endpoints PHP, configuracion de autenticacion, consultas SQL y renderizado JavaScript. No se realizaron pruebas destructivas.

Resumen ejecutivo

Se han identificado vulnerabilidades criticas relacionadas con control de acceso en endpoints de usuarios y reservas. La prioridad inmediata debe ser cerrar los endpoints sin autorizacion, retirar el bypass de desarrollo en entornos no locales y endurecer la gestion de sesion.

Prioridad recomendada

- **1:** Corregir api/reservas/update.php, api/usuarios/get.php y api/reservas/get.php con verificarAdmin() o filtros por usuario de sesion.
- **2:** Garantizar que IS_DEV no pueda activar bypass en produccion y regenerar la sesion al iniciar login.
- **3:** Escapar datos dinamicos antes de usar innerHTML o sustituir por textContent.
- **4:** Mover credenciales a variables de entorno, rotarlas y desactivar display_errors.
- **5:** Anadir CSRF, CORS restringido y transacciones en reservas.

Detalle de vulnerabilidades

Critica - Cambio de estado de reservas sin autorizacion

- **Evidencia:** api/reservas/update.php importa auth.php, pero no llama a verificarAdmin() ni verificarSesion(). El endpoint acepta id y estado por JSON y ejecuta UPDATE reserva SET estado = ? WHERE id = ?.
- **Impacto:** Un usuario no autenticado podria confirmar, cancelar o dejar pendiente cualquier reserva si conoce o prueba un id.
- **Solucion:** Llamar a verificarAdmin() despues de cargar auth.php y antes de leer el body. Mantener la lista blanca de estados y devolver 401/403 cuando no haya permisos. Si usuarios normales deben cambiar sus propias reservas, crear un endpoint separado que filtre por usuario_id de la sesion.

Critica - Listado de usuarios expuesto publicamente

- **Evidencia:** api/usuarios/get.php no carga auth.php y devuelve id, nombre, apellidos, email, telefono, foto_perfil, rol, activo y created_at.
- **Impacto:** Exposicion de datos personales y estructura de roles. Facilita phishing, enumeracion de usuarios y ataques dirigidos a administradores.
- **Solucion:** Importar auth.php y llamar a verificarAdmin(). Si alguna vista publica necesita usuarios, crear una respuesta publica minima sin emails, telefonos, rol ni estado.

Critica - Listado de reservas expuesto publicamente

- **Evidencia:** api/reservas/get.php no exige autenticacion y devuelve reservas con usuario_id, paquete_id, precio_total, estado, fecha_reserva, nombre, apellidos y email.

Informe de vulnerabilidades - Turistea

- **Impacto:** Filtración de reservas y datos personales de clientes. Puede revelar historiales de compra, importes y estados de viaje.
- **Solución:** Protegerlo con `verificarAdmin()` o sustituir su uso por `api/reservas/listar.php`, que ya verifica administrador. Para perfil de usuario, usar endpoints que filtren por `$_SESSION["usuario_id"]`.

Alta - Modo desarrollo puede saltarse toda la autenticación

- **Evidencia:** `api/config/auth.php` hace return en `verificarSesion()` y `verificarAdmin()` cuando `IS_DEV=true`. `api/auth/session.php` simula un usuario admin en modo dev.
- **Impacto:** Si `IS_DEV=true` llega a un servidor real, cualquier endpoint protegido queda accesible como administrador.
- **Solución:** Asegurar que `IS_DEV=false` en producción. Mejor aun: eliminar el bypass global o limitarlo a CLI/localhost comprobando `REMOTE_ADDR`. No simular administradores en endpoints HTTP compartidos.

Alta - Riesgo de fijación de sesión en login

- **Evidencia:** `api/auth/login.php` llama a `session_start()` y asigna variables de sesión tras `password_verify()`, pero no ejecuta `session_regenerate_id(true)`.
- **Impacto:** Un atacante que consiga fijar un `PHPSESSID` antes del login podría reutilizar la sesión autenticada.
- **Solución:** Ejecutar `session_regenerate_id(true)` inmediatamente después de verificar la contraseña y antes de asignar `usuario_id`, email, nombre y rol.

Alta - XSS almacenado o reflejado por uso de innerHTML con datos de BD

- **Evidencia:** `frontend/js/paquetes/tarjetas.js`, `frontend/js/paquetes/detalle.js`, `frontend/js/admin/viajes/viajesRender.js` y `frontend/js/admin/clientes/clientesRender.js` interpolan campos como título, descripción, destino, email o teléfono dentro de HTML.
- **Impacto:** Si un valor malicioso entra en la base de datos, puede ejecutar JavaScript en navegadores de clientes o administradores.
- **Solución:** Renderizar datos de usuario/BD con `textContent` o crear nodos DOM. Si se mantiene template HTML, escapar todos los valores dinámicos con una función `escapeHTML` antes de insertarlos.

Media - Credenciales en el código fuente

- **Evidencia:** `api/config/bd.php` contiene usuario y contraseña de base de datos. `api/config/smtp.php` contiene credenciales SMTP.
- **Impacto:** Cualquier copia del repositorio o backup expone accesos internos. Si se sube a un remoto, las credenciales deben considerarse comprometidas.
- **Solución:** Mover credenciales a `.env` o variables de entorno, añadir `.env` a `.gitignore` y rotar las claves ya expuestas. El código debe leer `getenv()/$_ENV` con valores por defecto solo locales.

Media - Errores PHP visibles en producción

- **Evidencia:** `api/config/bd.php` activa `ini_set("display_errors", 1)` y `error_reporting(E_ALL)`.
- **Impacto:** Los errores pueden revelar rutas internas, mensajes SQL, configuración y detalles útiles para ataques.
- **Solución:** Desactivar `display_errors` en producción y registrar errores en logs: `ini_set("display_errors", 0)`, `ini_set("log_errors", 1)`. Controlarlo por entorno.

Media - Validación insuficiente de tarjeta al crear reserva

Informe de vulnerabilidades - Turistea

- **Evidencia:** api/reservas/crear.php solo valida que tarjeta_id sea mayor que 0; no comprueba que exista ni que pertenezca al usuario autenticado.
- **Impacto:** Permite crear reservas indicando ids arbitrarios de tarjeta. Aunque solo se guardan ultimos 4 digitos, rompe la integridad de la logica de pago.
- **Solucion:** Consultar tarjeta_credito WHERE id = ? AND usuario_id = ? antes de crear la reserva. Si no existe, devolver 422/403.

Media - Posible sobreventa de plazas por condicion de carrera

- **Evidencia:** api/reservas/crear.php lee plazas_disponibles, inserta reserva y despues descuenta plazas en una consulta separada sin transaccion ni condicion plazas_disponibles >= num_viajeros.
- **Impacto:** Dos peticiones simultaneas pueden superar las plazas disponibles.
- **Solucion:** Usar transaccion. Descontar con UPDATE paquete SET plazas_disponibles = plazas_disponibles - ? WHERE id = ? AND plazas_disponibles >= ? y comprobar affected_rows antes de insertar o confirmar la reserva.

Media - CORS demasiado permisivo en endpoints sensibles

- **Evidencia:** Varios endpoints declaran Access-Control-Allow-Origin: *, incluyendo api/usuarios/get.php, api/reservas/get.php, api/reservas/update.php y api/auth/logout.php.
- **Impacto:** Amplia la superficie de abuso desde otros origenes. En endpoints con credenciales puede combinarse con malas configuraciones de cookies o navegadores.
- **Solucion:** Restringir Access-Control-Allow-Origin al dominio real de la aplicacion. Para endpoints autenticados, permitir credentials solo con origen explicito y no usar *.

Media - Falta proteccion CSRF en acciones con sesion

- **Evidencia:** Endpoints POST como usuarios/update.php, usuarios/delete.php, reservas/update.php, paquetes/create.php, paquetes/update.php y paquetes/delete.php confian solo en la cookie de sesion.
- **Impacto:** Si la cookie se envia automaticamente, un sitio externo podria intentar disparar acciones contra la sesion de un administrador.
- **Solucion:** Emitir token CSRF por sesion y exigirlo en una cabecera como X-CSRF-Token para operaciones POST/PUT/DELETE. Configurar cookies con SameSite=Lax o Strict, HttpOnly y Secure en HTTPS.

Media - Endpoint de contacto permite abuso y revela errores internos

- **Evidencia:** api/contacto/enviar.php no aplica rate limiting/captcha y devuelve detalles de ErrorInfo o excepciones SMTP al cliente.
- **Impacto:** Puede usarse para spam o pruebas masivas. Los mensajes de error pueden revelar detalles del proveedor SMTP o configuracion.
- **Solucion:** Aplicar limite por IP/email, captcha o honeypot, y devolver errores genericos al cliente. Enviar detalles solo a error_log.

Baja - SQL dinamico con escape manual

- **Evidencia:** api/usuarios/get.php y api/reservas/get.php construyen filtros SQL concatenando valores escapados con real_escape_string().
- **Impacto:** Aunque el escape reduce el riesgo, es mas facil introducir una inyeccion SQL al ampliar filtros o cambiar tipos.

Informe de vulnerabilidades - Turistea

- **Solucion:** Migrar filtros a prepared statements con bind_param. Mantener listas blancas para campos como estado, rol y ordenacion.

Baja - Logout con CORS inconsistente

- **Evidencia:** api/auth/logout.php combina Access-Control-Allow-Origin: * con Access-Control-Allow-Credentials: true.

- **Impacto:** Es una configuracion invalida/inconsistente para credenciales y puede causar comportamientos diferentes entre navegadores.

- **Solucion:** Usar un origen explicito cuando Allow-Credentials sea true, por ejemplo https://turistea.example, y responder Vary: Origin si se permite una lista de origenes.

Checklist de cierre

- **Control de acceso:** Todos los endpoints privados llaman a verificarSesion() o verificarAdmin() segun corresponda.

- **Sesiones:** Login regenera PHPSESSID; cookies usan HttpOnly, Secure y SameSite.

- **Datos:** No se devuelven datos personales en endpoints publicos.

- **Frontend:** Los campos de BD se pintan con textContent o escapeHTML.

- **Configuracion:** Credenciales fuera del repositorio, errores no visibles y CORS limitado al dominio permitido.

- **Reservas:** Creacion de reservas dentro de transaccion y descuento de plazas atomico.